Name – Jyotsna Kasibhotla
Roll no. – 23102B0078

COMMUNITY HEALTH SURVEY SYSTEM

DevOps Mini-Project — Week 2

Agile Planning and DevOps Workflow

1. User Stories and Acceptance Criteria

User stories follow the standard format: As a <role>, I want <capability>, so that <benefit>. Each story includes acceptance criteria that define when the story is considered complete.

US-1: Create a Survey Record

As a Surveyor, I want to create a new survey record, so that community health data is captured digitally instead of on paper.

- Given valid mandatory fields (name, age, gender, location, health condition, survey date), a new record is saved with status DRAFT.
- Given a missing mandatory field, the system shows a validation error and does not save the record.
- The created record stores the creator's user ID.

US-2: Search Survey Records

As a Surveyor or Health Officer, I want to search survey records by ID, name or another field, so that I can quickly find existing entries.

- Given a valid search term, matching records are displayed in a results list.
- Given no matches, the system shows a clear "no records found" message.
- Search is case-insensitive for name-based lookup.

US-3: Update Survey Status

As an Administrator, I want to change a survey's status, so that the record reflects its correct workflow stage.

- Only valid forward transitions are allowed: DRAFT → SUBMITTED → VERIFIED → CLOSED.
- An invalid transition (e.g., DRAFT → CLOSED directly) is rejected with an error message.
- Every status change is attributed to the administrator who made it.

US-4: View Dashboard Summary

As a Health Officer, I want to view a dashboard of survey counts by status, so that I can quickly assess overall progress.

- The dashboard shows total record count and a count for each status (DRAFT, SUBMITTED, VERIFIED, CLOSED).
- Counts update immediately after any record is created or its status changes.

US-5: Role-Based Login

As any user, I want to log in with a role-specific account, so that I only see the actions relevant to my role.

- Valid credentials grant access to a role-appropriate view (Surveyor / Administrator / Health Officer).
- Invalid credentials are rejected with a generic error message (no detail on which field was wrong).

US-6: Automated Regression via Selenium

As the project team, I want critical user journeys automated with Selenium, so that regressions are caught before deployment.

- At least 3–5 journeys (login, create record, search, status update, dashboard view) are automated.
- Selenium tests run via Maven locally and via Jenkins in CI.
- A failing Selenium test blocks the Jenkins pipeline from deploying.

2. Product Backlog

The backlog is prioritized using MoSCoW (Must / Should / Could) and mapped to the relevant week range.

ID	Backlog Item	Priority	Target Weeks
PB-1	Project skeleton (Spring Boot + Maven + MySQL)	Must	Week 3
PB-2	Git/GitHub repository, README, branching policy	Must	Week 4
PB-3	Create survey record (US-1)	Must	Week 5
PB-4	Search survey records (US-2)	Must	Week 5-6
PB-5	Status workflow engine (US-3)	Must	Week 6
PB-6	Dashboard summary counts (US-4)	Must	Week 6
PB-7	Role-based login (US-5)	Must	Week 6
PB-8	Jenkins CI job connected to GitHub + Maven	Must	Week 7
PB-9	Jenkinsfile with build/test/package/deploy stages	Must	Week 8
PB-10	Selenium test suite (US-6)	Must	Week 9
PB-11	Continuous testing gate in Jenkins	Must	Week 10
PB-12	Dockerfile and container lifecycle	Must	Week 11
PB-13	Jenkins + Docker CD (versioned image)	Must	Week 12
PB-14	Ansible playbook and inventory	Must	Week 13
PB-15	Automated provisioning + idempotency + rollback	Should	Week 14
PB-16	Final documentation, demo and viva prep	Must	Week 15
PB-17	Update record details after creation	Should	Week 5-6
PB-18	Export dashboard counts as simple report	Could	Stretch

3. Task Board (Kanban)

The team uses a five-column Kanban board — Backlog, To Do (current sprint), In Progress, In Review/Test, and Done — to track work. A snapshot from an early sprint is shown below.

Figure 1: Kanban task board layout

4. 15-Week Sprint Plan

The 15 weeks are grouped into five sprints, each ending with a usable increment and a working demo.

Figure 2: 15-week sprint grouping

Sprint	Weeks	Goal
Sprint 0 — Foundation	1-3	Freeze scope, plan the workflow, stand up the local project skeleton.
Sprint 1 — Core App & Git	4-6	Build core CRUD/search/workflow/dashboard with Git branching and PR practice.
Sprint 2 — CI & Testing	7-10	Stand up Jenkins CI, pipeline as code, and Selenium-based continuous testing.
Sprint 3 — Containerize & Provision	11-14	Dockerize the app, wire up Jenkins CD, and automate provisioning with Ansible.
Sprint 4 — Release	15	Final release, full documentation, demo and viva.

5. Definition of Done (DoD)

A backlog item or user story is considered Done only when all of the following are true:

- Code is committed to a feature branch and merged into development via a reviewed pull request.
- The feature builds successfully with Maven (mvn clean package) with no failing unit tests.
- Relevant acceptance criteria for the story are met and manually verified.
- Applicable Selenium journey(s) pass locally and in Jenkins.
- Jenkins pipeline completes build → test → package → deploy without manual intervention.
- No critical or high-severity defect remains open against the item.
- Documentation (README/architecture notes) is updated if the change affects setup or usage.
- The change does not break the Docker image build or Ansible playbook execution, where applicable.

6. DevOps Lifecycle Diagram

The end-to-end lifecycle links planning, source control, build, test, release, deployment, operations and feedback into one continuous loop.

Figure 3: DevOps lifecycle — from Plan to Operate, with feedback back into Plan

A failing test at any stage stops the pipeline before it reaches deployment, and dashboard/operational feedback flows back into planning for the next sprint.

7. Week 2 Summary

The backlog, Kanban task board, 15-week sprint plan, Definition of Done and DevOps lifecycle diagram documented above complete the Week 2 deliverable. Together they establish the working process the team will follow through Weeks 3-15: architecture and setup, Git-based feature development, Jenkins CI, Selenium-based continuous testing, Docker containerization, Ansible provisioning, and final release.